

AI ENGINEERING — WEEK 2

OOP, Functions, Errors & Git: The Complete Reading Guide

Classes and objects, inheritance and polymorphism, Pythonic dunder methods, decorators, exception handling done properly, and the Git workflow real teams use — end to end, at your own pace.

Companion reading to the Week 2 lecture slides. Lab exercises are not included here — see the separate lab handout.

Chapter 1 — Classes & Objects: The Foundations

By the end of this chapter you should be able to

- Explain the difference between a class and an instance
- Write a class with `__init__`, instance attributes, and methods
- Explain the difference between instance attributes and class attributes
- Read the error you get when you forget `self`

1.1 What a Class Actually Is

A **class** is a blueprint. It describes what something looks like and what it can do, but it isn't the thing itself. An **instance** is one specific object built from that blueprint. Think of a class as the blueprint for a house, and each instance as an actual house built from it — same layout, different address, different owner, different paint color.

```
class Robot:
    pass    # an empty class – valid, but useless on its own

my_robot = Robot()    # my_robot is now an INSTANCE of the Robot class
print(type(my_robot))    # <class '__main__.Robot'>
```

1.2 `__init__` and `self`

`__init__` is a special method that runs automatically the moment an instance is created. It's where you set up the instance's initial state. `self` refers to "this particular instance" — it's how a method knows which object's data it's working with.

```
class Robot:
    def __init__(self, name, battery=100):
        self.name = name        # instance attribute
        self.battery = battery  # instance attribute

r2 = Robot("R2")
c3 = Robot("C3", battery=50)

print(r2.name, r2.battery)    # R2 100
print(c3.name, c3.battery)    # C3 50
```

Why `self` has to be there: when you call `r2.name`, Python is really calling something like `Robot.name(r2)` behind the scenes — `self` is just the formal parameter that receives `r2`. Every instance method needs `self` as its first parameter so it knows which instance's data to read or modify. Forgetting it is one of the most common early OOP mistakes.

Common error — forgetting `self`:

```
class Robot:
    def __init__(name, battery=100):    # missing self!
        name = name

r2 = Robot("R2")
# TypeError: Robot.__init__() takes 1 positional argument but 2 were given
```

Python is trying to pass `self` automatically (that's `r2` itself), plus the argument you gave (`"R2"`) — that's two arguments arriving where your broken signature only expects one (`name` , which accidentally became the slot for `self`).

1.3 Instance Attributes vs. Class Attributes

An **instance attribute** belongs to one specific object (like `self.name` above). A **class attribute** is shared by every instance of the class — defined directly inside the class body, not inside `__init__`.

```
class Robot:
    manufacturer = "Cyberdyne"    # class attribute – shared by ALL robots

    def __init__(self, name):
        self.name = name          # instance attribute – unique per robot

r2 = Robot("R2")
c3 = Robot("C3")

print(r2.manufacturer)    # Cyberdyne
print(c3.manufacturer)    # Cyberdyne – same value, shared

Robot.manufacturer = "Tyrell Corp"    # change it on the CLASS
print(r2.manufacturer)    # Tyrell Corp – every instance sees the update
```

The mutable class attribute trap: just like the mutable default argument trap with functions, a mutable class attribute (a list or dict) is shared and can cause the exact same kind of unexpected accumulation across instances.

```
# DON'T do this
class Robot:
    logs = []    # shared by every instance!

    def add_log(self, entry):
        self.logs.append(entry)    # modifies the SHARED list

# DO this instead – build the list per-instance in __init__
class Robot:
    def __init__(self):
        self.logs = []    # each instance gets its own list
```

1.4 Methods

A method is just a function defined inside a class, which operates on a specific instance via `self`.

```
class Robot:
    def __init__(self, name, battery=100):
        self.name = name
        self.battery = battery

    def use_battery(self, amount):
        self.battery -= amount
        if self.battery < 0:
            self.battery = 0

    def status(self):
        return f"{self.name}: {self.battery}% battery"

r2 = Robot("R2")
r2.use_battery(30)
print(r2.status())    # R2: 70% battery
```

1.5 A Complete Worked Example

Putting it all together — a small but complete class you'll build on throughout this chapter:

```
class BankAccount:
    bank_name = "Python National Bank"    # class attribute

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient funds"
        self.balance -= amount
        return self.balance

account = BankAccount("Amara", balance=100)
account.deposit(50)
print(account.withdraw(30))    # 120
print(account.owner, account.balance)    # Amara 120
```

Chapter 2 — Inheritance & Polymorphism

By the end of this chapter you should be able to

- Create a child class that inherits from a parent class
- Use `super().__init__()` correctly
- Override a parent method and explain why that's useful (polymorphism)
- Explain the difference between inheritance and composition, and when to prefer each
- Explain duck typing and check types safely with `isinstance()`
- Build a multi-level inheritance chain and explain how `super()` cascades through it
- Use multiple inheritance and mixins, and explain what MRO resolves

2.1 Why Inheritance

Inheritance lets a new class reuse the behavior of an existing class, then add or change only what's different. This avoids retyping the same logic across similar classes.

```
class Robot:
    def __init__(self, name, battery=100):
        self.name = name
        self.battery = battery

    def use_battery(self, amount):
        self.battery -= amount
        if self.battery < 0:
            self.battery = 0

    def status(self):
        return f"{self.name}: {self.battery}% battery"

class CleaningRobot(Robot):  # CleaningRobot INHERITS from Robot
    pass

c1 = CleaningRobot("Roomba")
print(c1.status())  # Roomba: 100% battery – inherited, unchanged
```

2.2 `super().__init__()`

When a child class needs its own `__init__` — say, to add a new attribute — it should still call the parent's `__init__` to set up everything the parent handles, rather than duplicating that logic.

```
class CleaningRobot(Robot):
    def __init__(self, name, battery=100, dust_capacity=500):
        super().__init__(name, battery)  # let Robot handle name & battery
        self.dust_capacity = dust_capacity  # new attribute, specific to this subclass
        self.dust_collected = 0

    def clean(self, amount):
        self.dust_collected = min(self.dust_capacity, self.dust_collected + amount)
        self.use_battery(5)

r = CleaningRobot("Roomba", dust_capacity=300)
print(r.name, r.dust_capacity)  # Roomba 300
```

Common mistake: forgetting to call `super().__init__()` and manually rewriting the parent's setup logic instead. This works at first, but any future change to the parent's `__init__` silently won't apply to the child — `super()` keeps the two in sync automatically.

2.3 Overriding Methods — Polymorphism

A child class can redefine a method it inherited, replacing the parent's behavior entirely. This is called **overriding**, and it's the foundation of **polymorphism** — the idea that different classes can respond to the same method call in their own way.

```

class Robot:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"{self.name}: beep boop."

class ProtocolRobot(Robot):
    def greet(self): # OVERRIDES the parent's greet()
        return f"{self.name}: Greetings, I am fluent in over six million forms of communication."

robots = [Robot("R2"), ProtocolRobot("C3")]
for bot in robots:
    print(bot.greet())
# R2: beep boop.
# C3: Greetings, I am fluent in over six million forms of communication.

```

Notice the loop doesn't care what specific class each `bot` is — it just calls `.greet()` and each object responds in its own way. That's polymorphism in practice: you write code against the general shape (any `Robot`), and each subclass fills in its own specifics.

2.4 Extending an Overridden Method with `super()`

Sometimes you don't want to fully replace a parent method — you want to add to it. `super()` lets you call the parent's version and then add extra behavior around it.

```

class ProtocolRobot(Robot):
    def greet(self):
        original = super().greet() # get the parent's greeting first
        return original + " (translation services also available.)"

```

2.5 Inheritance vs. Composition

Inheritance answers "is this a type of that?" (a `CleaningRobot` *is a* `Robot`). **Composition** answers "does this have one of those?" — building a class out of other objects as attributes, rather than inheriting from them.

```

# Inheritance: CleaningRobot IS-A Robot
class CleaningRobot(Robot):
    ...

# Composition: Robot HAS-A Battery
class Battery:
    def __init__(self, capacity=100):
        self.capacity = capacity
        self.level = capacity

    def use(self, amount):
        self.level = max(0, self.level - amount)

class Robot:
    def __init__(self, name):
        self.name = name
        self.battery = Battery() # Robot is composed of a Battery, not a subclass of it

r = Robot("R2")
r.battery.use(30)
print(r.battery.level) # 70

```

Rule of thumb: reach for inheritance when a real "is-a" relationship exists and behavior should genuinely be shared. Reach for composition when you're tempted to inherit just to reuse some functionality, but the relationship isn't really "is-a" — composition tends to stay more flexible as a codebase grows, since swapping out a component (like a different `Battery` type) doesn't require restructuring the whole class hierarchy.

2.6 Encapsulation, Briefly

Encapsulation means keeping an object's internal details bundled together and controlling how the outside world interacts with them, rather than letting any code reach in and modify internal state freely. Python doesn't enforce strict private/protected access like some other languages — it relies on convention instead.

Convention	Meaning
<code>name</code>	Public — anyone can use it freely
<code>_name</code>	Convention for "internal use" — still accessible, but a signal to other developers not to touch it directly
<code>__name</code>	Name-mangled — Python renames it internally to discourage accidental access from outside the class

The single underscore convention is by far the most common in everyday Python code — it communicates intent without fighting the language.

2.7 Duck Typing & Checking Types Safely

Python's polymorphism generally doesn't rely on shared inheritance at all — it relies on **duck typing**: "if it walks like a duck and quacks like a duck, treat it like a duck." Any object with the right method will work, regardless of its class.

```
class Duck:
    def speak(self):
        return "Quack!"

class Robot:
    def speak(self):
        return "Beep!"

def make_it_speak(thing):
    return thing.speak()    # doesn't care what class 'thing' is — only that .speak() exists

print(make_it_speak(Duck()))    # Quack!
print(make_it_speak(Robot()))   # Beep! — Robot isn't related to Duck at all, and it still works
```

Sometimes you do need to check an object's type deliberately — for validation, or to branch behavior. `isinstance()` is the right tool, and it respects inheritance (a `CleaningRobot` instance is also a `Robot` instance):

```
class Robot:
    pass

class CleaningRobot(Robot):
    pass

c = CleaningRobot()
print(isinstance(c, CleaningRobot))    # True
print(isinstance(c, Robot))            # True — CleaningRobot IS-A Robot
print(isinstance(c, str))              # False

print(issubclass(CleaningRobot, Robot))    # True — checks the CLASS relationship, not an instance
```

Avoid `type(obj) == SomeClass` for this kind of check — it fails to recognize subclasses. `isinstance()` is almost always what you actually want.

2.8 Multi-Level Inheritance

A subclass can itself be the parent of another subclass — inheritance chains as deep as you need. Each level adds or overrides on top of everything above it.

```

class Robot:
    def __init__(self, name, battery=100):
        self.name = name
        self.battery = battery

    def status(self):
        return f"{self.name}: {self.battery}% battery"

class CleaningRobot(Robot):
    def __init__(self, name, battery=100, dust_capacity=500):
        super().__init__(name, battery)
        self.dust_capacity = dust_capacity
        self.dust_collected = 0

    def clean(self, amount):
        self.dust_collected = min(self.dust_capacity, self.dust_collected + amount)

class SmartCleaningRobot(CleaningRobot):  # grandchild of Robot, child of CleaningRobot
    def __init__(self, name, battery=100, dust_capacity=500):
        super().__init__(name, battery, dust_capacity)  # calls CleaningRobot's __init__,
        self.rooms_mapped = []                        # which in turn calls Robot's

    def map_room(self, room_name):
        self.rooms_mapped.append(room_name)

s = SmartCleaningRobot("Roomba X")
s.clean(50)
s.map_room("Kitchen")
print(s.status())      # Roomba X: 100% battery – inherited from Robot, two levels up
print(s.dust_collected)  # 50 – inherited from CleaningRobot, one level up
print(s.rooms_mapped)   # ['Kitchen'] – defined on SmartCleaningRobot itself

```

Each `super().__init__()` call only reaches up *one* level — but because every level in the chain calls it, the setup logic cascades all the way from `SmartCleaningRobot` up through `CleaningRobot` to `Robot` automatically. You never have to manually reach two levels up.

2.9 Multiple Inheritance, Mixins & MRO

Python allows a class to inherit from more than one parent at once — unlike some languages that only allow a single parent.

```

class Flyable:
    def fly(self):
        return f"{self.name} takes off!"

class Swimmable:
    def swim(self):
        return f"{self.name} dives in!"

class DroneRobot(Flyable, Swimmable):  # inherits from BOTH
    def __init__(self, name):
        self.name = name

d = DroneRobot("Aqua-Drone")
print(d.fly())  # Aqua-Drone takes off!
print(d.swim()) # Aqua-Drone dives in!

```

A class like `Flyable` here — small, focused, meant to be combined with other classes rather than used on its own — is called a **mixin**. This is a common, practical use of multiple inheritance: build small, single-purpose behaviors, then compose them onto a real class as needed, instead of building one deep, rigid inheritance chain.

The Diamond Problem & MRO

If two parents both define the same method name, which one wins? Python resolves this with a defined order called the **Method Resolution Order (MRO)** — roughly, it checks the class itself, then its parents left to right as listed, then their parents, and so on.

```
class A:
    def greet(self):
        return "Hello from A"

class B:
    def greet(self):
        return "Hello from B"

class C(A, B):    # lists A before B
    pass

print(C().greet())    # Hello from A – A comes first in the class definition
print(C.__mro__)      # shows the exact lookup order Python will follow
# (<class 'C'>, <class 'A'>, <class 'B'>, <class 'object'>)
```

Multiple inheritance is powerful but easy to overuse. If you find yourself checking `__mro__` to understand your own class, that's usually a sign the hierarchy has gotten more complicated than the problem needs. Prefer small, focused mixins (like `Flyable` above) over deep, tangled multiple-inheritance trees.

Chapter 3 — Writing Pythonic Classes

By the end of this chapter you should be able to

- Make an object print in a readable way with `__str__` and `__repr__`
- Explain the difference between an instance method, a static method, and a class method
- Use `@property` to control attribute access
- Implement basic operator overloading with `__eq__`
- Use an abstract base class to enforce that subclasses implement a method

3.1 Making Objects Debuggable — `__str__` and `__repr__`

Without help, printing an object gives you something unreadable:

```
class Robot:
    def __init__(self, name):
        self.name = name

r = Robot("R2")
print(r)    # <__main__.Robot object at 0x7f9a2c0b5d90>
```

`__str__` defines what `print()` and `str()` show — meant for humans. `__repr__` defines the "official," unambiguous representation — meant for developers and debugging (shown in a REPL, or when an object appears inside a list).

```
class Robot:
    def __init__(self, name, battery=100):
        self.name = name
        self.battery = battery

    def __str__(self):
        return f"{self.name} ({self.battery}% battery)"

    def __repr__(self):
        return f"Robot(name={self.name!r}, battery={self.battery!r})"

r = Robot("R2")
print(r)        # R2 (100% battery) — uses __str__
print([r])      # [Robot(name='R2', battery=100)] — uses __repr__
```

Rule of thumb: always write `__repr__` for your classes — it's what shows up by default in error messages and debugging tools. Write `__str__` as well when you want a friendlier, human-facing version specifically for `print()`. If you only define one, Python falls back to `__repr__` for both.

3.2 Instance Methods vs. Static Methods vs. Class Methods

Not every method needs access to a specific instance's data.

Type	First parameter	Use it when...
Instance method	<code>self</code>	The method needs to read or change this specific instance's data (the vast majority of methods)
<code>@staticmethod</code>	none required	The logic is related to the class conceptually, but doesn't need <code>self</code> or the class itself at all
<code>@classmethod</code>	<code>cls</code>	The method needs the class itself — most often used for alternative constructors

```

class Robot:
    population = 0

    def __init__(self, name):
        self.name = name
        Robot.population += 1

    @staticmethod
    def is_valid_name(name):
        return len(name) > 0 and name.isalnum()

    @classmethod
    def from_config(cls, config_dict):
        return cls(config_dict["name"]) # an alternative way to construct a Robot

print(Robot.is_valid_name("R2")) # True – no instance needed to check this

r = Robot.from_config({"name": "C3"}) # built via the classmethod instead of Robot("C3")
print(r.name, Robot.population)      # C3 1

```

3.3 @property — Controlled Attribute Access

`@property` lets a method be accessed like a plain attribute (no parentheses), which is useful for computed values or for adding validation without breaking the simple `object.attribute` syntax callers expect.

```

class Robot:
    def __init__(self, name, battery=100):
        self.name = name
        self._battery = battery # leading underscore: internal storage

    @property
    def battery(self):
        return self._battery

    @battery.setter
    def battery(self, value):
        if value < 0:
            value = 0
        if value > 100:
            value = 100
        self._battery = value

r = Robot("R2")
r.battery = 150 # goes through the setter – gets clamped
print(r.battery) # 100 – accessed like a plain attribute, no parentheses

```

This is a very common pattern: store the real value under a "private" name (`_battery`), then expose a clean public name (`battery`) through `@property` that validates or computes on the way in and out.

3.4 Basic Operator Overloading — `__eq__`

By default, comparing two instances with `==` checks identity (same as `is`), not whether their data matches:

```

class Robot:
    def __init__(self, name):
        self.name = name

r1 = Robot("R2")
r2 = Robot("R2")
print(r1 == r2) # False – different objects in memory, even with the same name

```

Defining `__eq__` lets you decide what "equal" means for your class:

```

class Robot:
    def __init__(self, name):
        self.name = name

    def __eq__(self, other):
        return self.name == other.name

r1 = Robot("R2")
r2 = Robot("R2")
print(r1 == r2)    # True — now comparing by name

```

There's a whole family of these "dunder" (double-underscore) methods — `__len__` for supporting `len(obj)`, `__lt__` for `<`, `__add__` for `+`, and more. You won't need most of them right away, but recognizing the pattern — Python calls a specific dunder method behind the scenes for built-in syntax — means you'll know where to look when you need one.

3.5 Abstract Base Classes

Duck typing works well informally, but sometimes you want to **enforce** that every subclass implements a particular method — catching the mistake at the moment a broken subclass is created, not later when something calls a method that was never written. That's what an abstract base class does.

```

from abc import ABC, abstractmethod

class Robot(ABC):
    @abstractmethod
    def move(self):
        """Every subclass MUST implement this."""
        pass

class WheeledRobot(Robot):
    def move(self):
        return "Rolling forward"

w = WheeledRobot()
print(w.move())    # Rolling forward

r = Robot()        # TypeError: Can't instantiate abstract class Robot with abstract method move

```

`Robot` itself can never be instantiated directly once it inherits from `ABC` and has an `@abstractmethod` — it exists purely to define a contract that subclasses must fulfill. This is especially useful once a codebase has many subclasses and you want a guarantee, checked automatically, that none of them forgot to implement something essential.

Chapter 4 — Functions as First-Class Citizens & Decorators

By the end of this chapter you should be able to

- Explain what it means for functions to be "first-class" in Python
- Pass a function as an argument to another function
- Read and write a simple decorator
- Explain what recursion is and when it's the right tool
- Explain closures, use `functools.wraps`, and write a basic generator with `yield`

4.1 Functions Are Values

In Python, a function is a value like any other — it can be assigned to a variable, stored in a list, or passed into another function.

```
def shout(text):
    return text.upper() + "!"

my_func = shout          # no parentheses – we're assigning the FUNCTION itself, not calling it
print(my_func("hello"))  # HELLO!

def apply_twice(func, value):
    return func(func(value))

print(apply_twice(shout, "hi"))  # HI!!
```

4.2 Higher-Order Functions

A function that accepts another function as an argument (or returns one) is called a **higher-order function**. You've already used these — `sorted(..., key=...)` and `map() / filter()` both take a function as an argument.

4.3 Decorators

A decorator is a function that wraps another function, adding behavior before or after it runs, without changing the original function's code. You've already been *using* decorators (`@staticmethod`, `@classmethod`, `@property`) — this section shows what's actually happening underneath.

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} finished")
        return result
    return wrapper

@log_call
def greet(name):
    return f"Hello, {name}!"

print(greet("Amara"))
# Calling greet
# greet finished
# Hello, Amara!
```

`@log_call` above `def greet(...)` is just shorthand for `greet = log_call(greet)`. Once you see a decorator as "a function that takes a function and returns a new, wrapped function," the syntax stops feeling like magic.

Closures — Why `wrapper` Can See `func`

`wrapper` can reference `func` even after `log_call` has already finished running — this works because `wrapper` is a **closure**: an inner function that "remembers" the variables from the scope it was created in, even after that outer function has returned.

```
def make_multiplier(factor):
    def multiply(n):
        return n * factor    # 'factor' is remembered from the enclosing scope
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5))    # 10 – remembers factor=2
print(triple(5))    # 15 – remembers factor=3, a completely separate "memory"
```

functools.wraps — Don't Skip This

There's a subtle cost to writing a decorator the way `log_call` was written above: the wrapped function loses its original identity.

```
print(greet.__name__)    # 'wrapper' – not 'greet'! Debugging tools now see the wrong name.
```

`functools.wraps` fixes this by copying over the original function's name, docstring, and other metadata onto the wrapper:

```
from functools import wraps

def log_call(func):
    @wraps(func)    # preserves greet's real name and docstring
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_call
def greet(name):
    return f"Hello, {name}!"

print(greet.__name__)    # 'greet' – fixed
```

Skipping `@wraps` is an easy mistake that only bites later — error messages, documentation tools, and debuggers all rely on a function's real `__name__`, and a decorator without `@wraps` quietly corrupts it everywhere that decorator is used.

4.4 Recursion, Briefly

A recursive function calls itself, working toward a **base case** that stops the recursion. It's a natural fit for problems that break down into smaller versions of themselves.

```
def factorial(n):
    if n <= 1:            # base case – stops the recursion
        return 1
    return n * factorial(n - 1)    # recursive case – calls itself with a smaller problem

print(factorial(5))    # 120
```

Forgetting the base case causes infinite recursion, eventually crashing with a `RecursionError`. Every recursive function needs a condition that stops it, and every recursive call needs to move measurably closer to that condition.

4.5 Generators

A normal function computes everything and returns it all at once. A **generator** produces values one at a time, on demand, using `yield` instead of `return` — useful when a sequence is large, or even infinite, and you don't want to build the whole thing in memory upfront.

```
def countdown(n):
    while n > 0:
        yield n    # pauses here, hands back n, and resumes right after on the next call
        n -= 1

for number in countdown(3):
    print(number)

# 3
# 2
# 1
```

Calling a generator function doesn't run its body immediately — it returns a generator object that runs lazily, one step per

iteration:

```
gen = countdown(3)
print(gen)          # <generator object countdown at 0x...>
print(next(gen))    # 3 – runs up to the first yield
print(next(gen))    # 2 – resumes right where it left off
```

A list comprehension builds the entire list in memory immediately. The equivalent **generator expression** — same syntax, but with parentheses instead of square brackets — produces values lazily instead: `(x**2 for x in range(1000000))` takes almost no memory until you actually iterate over it, unlike `[x**2 for x in range(1000000)]`, which builds all million values right away.

Chapter 5 — Exception Handling, Properly

By the end of this chapter you should be able to

- Use the full `try / except / else / finally` structure correctly
- Catch specific exceptions instead of using a bare `except`
- Write and raise a custom exception
- Use context managers for resource cleanup
- Explain the EAFP principle and when it fits Python better than checking first
- Read the built-in exception hierarchy and use the `logging` module instead of debug prints

5.1 The Exception Hierarchy

Every exception in Python is a class, and they're organized in an inheritance tree. This is exactly why `except Exception:` catches almost everything, and why a bare `except:` is even more dangerous than it looks — it reaches past `Exception` all the way up to `BaseException`, which also covers things like `KeyboardInterrupt` and `SystemExit` that you almost never want to silently swallow.

```
BaseException
├── SystemExit
├── KeyboardInterrupt
├── Exception
│   ├── ArithmeticError
│   │   └── ZeroDivisionError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   ├── ValueError
│   ├── TypeError
│   ├── AttributeError
│   └── ... and many more
```

```
print(issubclass(ZeroDivisionError, ArithmeticError))    # True
print(issubclass(ZeroDivisionError, Exception))         # True – this is why except Exception catches it
print(issubclass(KeyboardInterrupt, Exception))         # False – this is why bare except: is riskier still
```

This is also why the order of `except` clauses matters: catch more specific exceptions before more general ones. If you catch `Exception` first, a later `except ZeroDivisionError:` clause below it will never run, since `Exception` already matched.

5.2 The Full `try` Structure

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Can't divide by zero")
else:
    print("This runs only if NO exception occurred")
finally:
    print("This always runs, error or not")

# Can't divide by zero
# This always runs, error or not
```

Block	Runs when...
<code>try</code>	Always attempted first
<code>except</code>	Only if a matching exception was raised inside <code>try</code>
<code>else</code>	Only if <code>try</code> completed with no exception at all
<code>finally</code>	Always — whether an exception happened or not, even if the exception was never caught

5.3 Catching Specific Exceptions

Catch the exception types you actually expect and know how to handle — not everything.

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Can't divide by zero"
    except TypeError:
        return "Both arguments must be numbers"

print(safe_divide(10, 0))    # Can't divide by zero
print(safe_divide(10, "x")) # Both arguments must be numbers
```

If two exception types should be handled identically, catch them together in one clause using a tuple, rather than repeating the same handling code twice:

```
def safe_divide(a, b):
    try:
        return a / b
    except (ZeroDivisionError, TypeError):
        return "Invalid division"
```

Why a bare `except:` is bad practice:

```
# DON'T do this
try:
    risky_operation()
except:
    pass # silently swallows EVERY error, including ones you never anticipated
```

A bare `except` also catches things like `KeyboardInterrupt` (Ctrl+C) and genuine bugs you'd actually want to know about. Catch specific exception types, or at minimum use `except Exception as e:` and log or print `e` so the failure isn't silent.

5.4 Raising Your Own Exceptions

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount

try:
    withdraw(100, 500)
except ValueError as e:
    print(f"Transaction failed: {e}")
# Transaction failed: Insufficient funds
```

5.5 Custom Exception Classes

For anything beyond a generic message, define your own exception by inheriting from `Exception` (or a more specific built-in exception).

```
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the available balance."""
    def __init__(self, balance, amount):
        message = f"Tried to withdraw {amount} but balance is only {balance}"
        super().__init__(message)
        self.balance = balance
        self.amount = amount

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount

try:
    withdraw(100, 500)
except InsufficientFundsError as e:
    print(e)
    print(e.amount) # you can still access the extra data you attached
# Tried to withdraw 500 but balance is only 100
# 500
```

Custom exceptions make error handling upstream far more precise — calling code can catch `InsufficientFundsError` specifically, rather than a generic `ValueError` that might mean several different things across a larger codebase.

5.6 Re-raising and Exception Chaining

Sometimes you want to catch an error, do something (like logging it), and then let it continue propagating rather than swallowing it:

```
try:
    withdraw(100, 500)
except InsufficientFundsError as e:
    print("Logging the error before re-raising...")
    raise # re-raises the SAME exception, unchanged
```

Or wrap a low-level exception in a more meaningful one, while preserving the original as context:

```
try:
    balance = int("not a number")
except ValueError as original_error:
    raise RuntimeError("Failed to process account balance") from original_error
# the traceback will show both errors, with a note that the second was
# "the direct cause" of the first — very useful for debugging
```

5.7 Context Managers — the `with` Statement

Some resources (files, network connections, locks) need to be reliably cleaned up whether or not an error occurs. `try / finally` can do this manually, but `with` does it automatically and more readably.

```
# the manual way
f = open("notes.txt", "w")
try:
    f.write("hello")
finally:
    f.close() # you have to remember this, even if writing failed

# the idiomatic way
with open("notes.txt", "w") as f:
    f.write("hello")
# f is automatically closed when the block ends, even if an error occurred inside it
```

5.8 `assert` Statements

`assert` checks that a condition holds, and raises `AssertionError` immediately if it doesn't. It's meant for catching your own bugs during development — not for validating user input or handling expected error conditions in production code.

```
def calculate_average(nums):
    assert len(nums) > 0, "Can't average an empty list"
    return sum(nums) / len(nums)

calculate_average([]) # AssertionError: Can't average an empty list
```

Don't use `assert` for things that can legitimately go wrong at runtime — like invalid user input or a missing file. Use proper exceptions (`raise ValueError(...)` , etc.) for those. Also note: assertions can be stripped out entirely when Python runs in optimized mode (`-O`), so never rely on an `assert` for logic your program actually needs to run correctly.

5.9 EAFP vs. LBYL

Python culture generally favors **EAFP** — "Easier to Ask Forgiveness than Permission" — over **LBYL** — "Look Before You Leap."

```
# LBYL – check first, then act
if "key" in my_dict:
    value = my_dict["key"]
else:
    value = None

# EAFP – just try it, and handle the failure if it happens
try:
    value = my_dict["key"]
except KeyError:
    value = None
```

EAFP is often preferred in Python because it avoids a subtle race condition (checking, then acting, leaves a gap where things could change in between — more relevant in concurrent code) and because it reads cleanly for the common case. That said, `.get()` is arguably cleaner than either for this specific dictionary example — EAFP shines most when the check itself would be as much work as just trying the operation.

5.10 The `logging` Module — Beyond `print()`

`print()` is fine for quick, throwaway debugging, but it doesn't scale: you can't easily turn it off, filter by severity, or send it somewhere other than the terminal. The `logging` module solves all three.

```
import logging

logging.basicConfig(level=logging.INFO)

def withdraw(balance, amount):
    if amount > balance:
        logging.error(f"Withdrawal of {amount} exceeds balance of {balance}")
        raise ValueError("Insufficient funds")
    logging.info(f"Withdrew {amount}, new balance: {balance - amount}")
    return balance - amount

withdraw(100, 500)
# ERROR:root:Withdrawal of 500 exceeds balance of 100
# (then the ValueError is still raised as normal – logging doesn't replace exception handling)
```

Level	Use for
DEBUG	Detailed diagnostic info, useful only while actively debugging
INFO	Confirmation that things are working as expected
WARNING	Something unexpected, but the program can continue
ERROR	A real problem — something failed
CRITICAL	A serious error — the program may not be able to continue

Setting `level=logging.INFO` shows `INFO` and everything more severe, but hides `DEBUG` messages — so you can leave detailed debug logging in your code permanently and just turn its visibility on or off by changing one line, instead of adding and deleting `print()` statements every time.

Chapter 6 — Git Fundamentals

By the end of this chapter you should be able to

- Explain what problem version control solves
- Initialize a repository and make commits
- Check status, view history, and see uncommitted changes
- Set up a `.gitignore` properly
- Tag a release and investigate history with `git log` filters and `git blame`

6.1 What Version Control Solves

Git tracks every change to your code over time, lets you jump back to any previous state, and lets multiple people work on the same project without overwriting each other's work. Without it, "final_v2_ACTUALLY_final.py" is the alternative.

6.2 The Core Workflow

```
# turn a folder into a git repository
git init

# check what's changed since the last commit
git status

# stage a specific file for the next commit
git add filename.py

# stage everything that's changed
git add .

# commit the staged changes with a message
git commit -m "Add user authentication"

# view commit history
git log

# a more compact one-line-per-commit view
git log --oneline
```

The three-stage mental model: your **working directory** (the actual files you're editing) → the **staging area** (changes you've marked as ready with `git add`) → the **repository** (permanently recorded once you `git commit`). Understanding this separation explains why `git add` exists at all — it lets you commit only part of what you've changed.

6.3 Seeing What Changed — `git diff`

```
git diff          # unstaged changes – what you've edited but not yet added
git diff --staged # staged changes – what you've added but not yet committed
```

6.4 Writing Good Commit Messages

A commit message should explain *what* changed and, when it's not obvious, *why*. A common convention: a short summary line (under ~50 characters, imperative mood), optionally followed by a blank line and more detail.

Weak	Better
fixed stuff	Fix off-by-one error in pagination
update	Add input validation to signup form
asdf	Remove unused import in utils.py

6.5 `.gitignore`

Certain files should never be committed — generated files, secrets, virtual environments, editor clutter. A `.gitignore` file tells Git to ignore them entirely.

```
# .gitignore
__pycache__/
*.pyc
.venv/
.env
.DS_Store
*.log
```

If you already committed something before adding it to `.gitignore`, adding it there won't remove it from history — Git is still tracking it. You need `git rm --cached filename` to untrack it going forward (the file stays on disk, it's just no longer tracked by Git).

6.6 Tags — Marking Releases

A tag marks a specific commit permanently — most commonly used to mark release versions (`v1.0.0` , `v2.1.3`), so you can always find or return to exactly what shipped.

```
git tag v1.0.0           # tag the current commit
git tag                  # list all tags
git push origin v1.0.0    # tags aren't pushed automatically – push them explicitly
git push origin --tags    # or push all tags at once
```

6.7 Investigating History — `git log` Filters & `git blame`

`git log` accepts filters that make it genuinely useful for investigation, not just a wall of commits:

```
git log --author="Amara"    # only commits by a specific person
git log --since="2 weeks ago" # commits after a given date
git log -p filename.py      # show the actual diff for each commit touching this file
git log --oneline --graph    # compact view with a visual branch graph
```

`git blame` shows who last changed each line of a file, and in which commit — invaluable when you find a confusing line and want to know the history behind it:

```
git blame filename.py
```

Chapter 7 — Branching, Remotes & Collaboration

By the end of this chapter you should be able to

- Create, switch between, and merge branches
- Connect a local repository to GitHub and push/pull changes
- Resolve a merge conflict
- Explain the pull request / code review workflow
- Use `git stash` to shelve work temporarily
- Explain rebase vs. merge, recognize detached HEAD state, and describe the fork-based contribution workflow

7.1 Why Branches

A branch is an independent line of development. It lets you work on a new feature or fix without touching the stable, working version of your code until you're ready to merge it back in.

```
git branch          # list branches
git branch feature-login  # create a new branch
git checkout feature-login  # switch to it
git checkout -b feature-login # create AND switch in one step

# modern alternative (Git 2.23+)
git switch feature-login
git switch -c feature-login  # create and switch
```

7.2 Merging

```
git checkout main          # switch to the branch you want to merge INTO
git merge feature-login    # bring feature-login's changes into main
```

7.3 Rebasing — An Alternative to Merging

`merge` combines two branches by creating a new merge commit that ties both histories together. `rebase` takes a different approach: it replays your branch's commits on top of the latest version of the branch you're rebasing onto, producing a straight, linear history instead of a merge commit.

```
git checkout feature-login
git rebase main          # replay feature-login's commits on top of main's latest commit
```

The one hard rule of rebasing: never rebase commits that have already been pushed and that someone else might have pulled. Rebase rewrites commit history — anyone who already has the old commits will end up with a confusing, diverged history that's genuinely painful to reconcile. Rebase freely on your own local, not-yet-pushed work; use merge (or ask before rebasing) once anything is shared.

7.4 Resolving a Merge Conflict

A conflict happens when Git can't automatically decide how to combine two changes to the same lines. Git marks the conflicting section directly in the file:

```
<<<<<<< HEAD
def greet():
    return "Hello!"
=====
def greet():
    return "Hi there!"
>>>>>>> feature-login
```

To resolve it: edit the file to keep the version you want (or a combination), remove the `<<<<<<< / ===== / >>>>>>>` marker lines entirely, then stage and commit the resolved file:

```
git add resolved_file.py
git commit -m "Resolve merge conflict in greet()"
```

7.5 Remotes — Connecting to GitHub

```
# first time connecting a local repo to a new GitHub repo
git remote add origin https://github.com/username/repo-name.git
git push -u origin main      # -u sets the upstream, so future pushes can just be `git push`

# getting someone else's repo onto your machine
git clone https://github.com/username/repo-name.git

# day-to-day
git push                    # send your commits to the remote
git pull                    # fetch and merge the remote's latest changes
```

Detached HEAD state: if you ever run `git checkout <some-commit-hash>` directly (rather than a branch name), Git puts you in "detached HEAD" state — you're looking at that specific commit, but you're not on any branch. You can look around safely, but any new commits made here can be easily lost once you switch away. If you want to keep changes made in this state, create a branch right where you are: `git checkout -b new-branch-name`. If you just wanted to look, `git checkout main` (or whatever branch you were on) gets you back.

7.6 Pull Requests & Code Review

A pull request (PR) is how teams merge work in practice: rather than merging straight into `main` locally, you push your branch to GitHub and open a PR asking for it to be reviewed and merged. Teammates comment on specific lines, request changes, and approve once it looks good — then the PR is merged, often by a "squash and merge" or "merge commit" button rather than the command line.

This is why real teams almost never commit straight to `main` — a feature branch plus a PR gives every change a review step and a paper trail before it reaches the code everyone else builds on.

Contributing to a Repo You Don't Own — Forking

If you don't have push access to a repository (common for open-source projects, or class assignments hosted as a shared repo), the workflow adds one extra step: **fork** the repository on GitHub (this creates your own copy under your account), clone *your fork*, make your changes on a branch, push to your fork, then open a PR from your fork's branch back to the original repository.

7.7 `git stash`

If you need to switch branches but aren't ready to commit your current changes, `stash` shelves them temporarily:

```
git stash          # shelve your uncommitted changes
git checkout other-branch
# ... do something else ...
git checkout original-branch
git stash pop      # bring your shelved changes back
```

Chapter 8 — Undoing Mistakes in Git

By the end of this chapter you should be able to

- Discard uncommitted changes safely
- Undo a commit without rewriting shared history
- Amend the most recent commit
- Explain why rewriting published history is dangerous

8.1 Discarding Uncommitted Changes

```
git checkout -- filename.py      # discard changes to one file, reverting to last commit
git restore filename.py         # modern equivalent (Git 2.23+)

git reset --hard                # discard ALL uncommitted changes in the working directory
# use with caution — this is not recoverable
```

8.2 Amending the Last Commit

Made a typo in your last commit message, or forgot to include a file? Amend it instead of making a whole new commit:

```
git add forgotten_file.py
git commit --amend -m "Corrected commit message"
```

Only amend commits that haven't been pushed yet, or that nobody else has already pulled. Amending rewrites the commit's identity — if someone else already has the old version, you'll create a confusing divergence.

8.3 `git revert` VS. `git reset`

Command	What it does	Safe on shared/pushed history?
<code>git revert</code> <code><commit></code>	Creates a new commit that undoes the changes from an earlier one — history is preserved	Yes
<code>git reset --hard</code> <code><commit></code>	Moves the branch pointer backward and discards commits after it entirely	No — never on a branch others have already pulled

```
git log --oneline                # find the commit hash you want to undo
git revert alb2c3d              # safely undoes that commit's changes with a new commit
```

Rule of thumb: if a commit has already been pushed and others might have it, use `revert`, not `reset`. Rewriting history that other people have already built on top of forces everyone else to untangle a mismatch between their history and yours — `revert` avoids that entirely by adding new history instead of erasing old history.

Before You Move On — Self-Check

Use this list to confirm you're ready to move forward. If any of these feel shaky, revisit that section before continuing.

- ☐ I can write a class with `__init__`, instance attributes, class attributes, and methods
- ☐ I understand why `self` is required and what error appears when it's missing
- ☐ I can create a child class, use `super().__init__()`, and override a method
- ☐ I can explain inheritance vs. composition and choose between them
- ☐ I can explain duck typing and check types safely with `isinstance()` / `issubclass()`
- ☐ I can build a multi-level inheritance chain and explain how `super()` cascades through it
- ☐ I can use multiple inheritance and mixins, and explain what MRO resolves and why deep hierarchies are risky
- ☐ I can implement `__str__`, `__repr__`, and `__eq__`
- ☐ I know when to use an instance method vs. `@staticmethod` vs. `@classmethod`
- ☐ I can use `@property` to validate or compute an attribute
- ☐ I can use an abstract base class to enforce that subclasses implement a method
- ☐ I can pass a function as an argument and explain what a decorator does, including what a closure is
- ☐ I know why `functools.wraps` matters when writing a decorator
- ☐ I can write a recursive function with a proper base case
- ☐ I can write a basic generator with `yield` and explain why it's more memory-efficient than building a full list
- ☐ I can read the built-in exception hierarchy and explain why catch order matters
- ☐ I can use the full `try / except / else / finally` structure, catch specific exceptions, and catch multiple types in one clause
- ☐ I can write and raise a custom exception class
- ☐ I can re-raise an exception and chain one exception to another with `raise ... from`
- ☐ I can use a `with` statement for resource cleanup
- ☐ I understand `assert`'s purpose and its limits
- ☐ I can explain EAFP vs. LBYL
- ☐ I can use the `logging` module instead of relying on `print()` for debugging
- ☐ I can initialize a repo, commit, check status/diff/log, and set up `.gitignore`
- ☐ I can tag a release and investigate history with `git log` filters and `git blame`
- ☐ I can create branches, merge them, and resolve a merge conflict
- ☐ I can explain rebase vs. merge and know the rule about not rebasing shared history
- ☐ I can connect to GitHub with remotes, recognize detached HEAD state, and explain the pull request and fork workflows
- ☐ I can stash and restore uncommitted work
- ☐ I can safely undo mistakes with `revert`, `amend`, and `restore`, and I know why `reset --hard` is dangerous on shared history